

unlike conjugate gradients, the success of the approach is not heavily dependent on the line search finding a point very close to the true minimum along the line. Thus, relative to conjugate gradients, BFGS has the advantage that it can spend less time refining each line search. On the other hand, the BFGS algorithm must store the inverse Hessian matrix, $\mathbf{M}$, that requires $O(n^2)$ memory, making BFGS impractical for most modern deep learning models that typically have millions of parameters.

Limited Memory BFGS (or L-BFGS) The memory costs of the BFGS algorithm can be significantly decreased by avoiding storing the complete inverse Hessian approximation $\mathbf{M}$. The L-BFGS algorithm computes the approximation $\mathbf{M}$ using the same method as the BFGS algorithm, but beginning with the assumption that $\mathbf{M}^{(t-1)}$ is the identity matrix, rather than storing the approximation from one step to the next. If used with exact line searches, the directions defined by L-BFGS are mutually conjugate. However, unlike the method of conjugate gradients, this procedure remains well behaved when the minimum of the line search is reached only approximately. The L-BFGS strategy with no storage described here can be generalized to include more information about the Hessian by storing some of the vectors used to update $\mathbf{M}$ at each time step, which costs only $O(n)$ per step.

8.7 Optimization Strategies and Meta-Algorithms

Many optimization techniques are not exactly algorithms, but rather general templates that can be specialized to yield algorithms, or subroutines that can be incorporated into many different algorithms.

8.7.1 Batch Normalization

Batch normalization (Ioffe and Szegedy, 2015) is one of the most exciting recent innovations in optimizing deep neural networks and it is actually not an optimization algorithm at all. Instead, it is a method of adaptive reparametrization, motivated by the difficulty of training very deep models.

Very deep models involve the composition of several functions or layers. The gradient tells how to update each parameter, under the assumption that the other layers do not change. In practice, we update all of the layers simultaneously. When we make the update, unexpected results can happen because many functions composed together are changed simultaneously, using updates that were computed under the assumption that the other functions remain constant. As a simple

example, suppose we have a deep neural network that has only one unit per layer and does not use an activation function at each hidden layer: $\hat{y} = xw_1w_2w_3 \dots w_l$. Here, w_i provides the weight used by layer i . The output of layer i is $h_i = h_{i-1}w_i$. The output $\hat{y}$ is a linear function of the input x , but a nonlinear function of the weights w_i . Suppose our cost function has put a gradient of 1 on $\hat{y}$, so we wish to decrease $\hat{y}$ slightly. The back-propagation algorithm can then compute a gradient $\mathbf{g} = \nabla_{\mathbf{w}}\hat{y}$. Consider what happens when we make an update $\mathbf{w} \leftarrow \mathbf{w} - \epsilon\mathbf{g}$. The first-order Taylor series approximation of $\hat{y}$ predicts that the value of $\hat{y}$ will decrease by $\epsilon\mathbf{g}^\top\mathbf{g}$. If we wanted to decrease $\hat{y}$ by .1, this first-order information available in the gradient suggests we could set the learning rate ϵ to $\frac{.1}{\mathbf{g}^\top\mathbf{g}}$. However, the actual update will include second-order and third-order effects, on up to effects of order l . The new value of $\hat{y}$ is given by

$$x(w_1 - \epsilon g_1)(w_2 - \epsilon g_2) \dots (w_l - \epsilon g_l). \quad (8.34)$$

An example of one second-order term arising from this update is $\epsilon^2 g_1 g_2 \prod_{i=3}^l w_i$. This term might be negligible if $\prod_{i=3}^l w_i$ is small, or might be exponentially large if the weights on layers 3 through l are greater than 1. This makes it very hard to choose an appropriate learning rate, because the effects of an update to the parameters for one layer depends so strongly on all of the other layers. Second-order optimization algorithms address this issue by computing an update that takes these second-order interactions into account, but we can see that in very deep networks, even higher-order interactions can be significant. Even second-order optimization algorithms are expensive and usually require numerous approximations that prevent them from truly accounting for all significant second-order interactions. Building an n -th order optimization algorithm for $n > 2$ thus seems hopeless. What can we do instead?

Batch normalization provides an elegant way of reparametrizing almost any deep network. The reparametrization significantly reduces the problem of coordinating updates across many layers. Batch normalization can be applied to any input or hidden layer in a network. Let $\mathbf{H}$ be a minibatch of activations of the layer to normalize, arranged as a design matrix, with the activations for each example appearing in a row of the matrix. To normalize $\mathbf{H}$, we replace it with

$$\mathbf{H}' = \frac{\mathbf{H} - \boldsymbol{\mu}}{\boldsymbol{\sigma}}, \quad (8.35)$$

where $\boldsymbol{\mu}$ is a vector containing the mean of each unit and $\boldsymbol{\sigma}$ is a vector containing the standard deviation of each unit. The arithmetic here is based on broadcasting the vector $\boldsymbol{\mu}$ and the vector $\boldsymbol{\sigma}$ to be applied to every row of the matrix $\mathbf{H}$. Within each row, the arithmetic is element-wise, so $H_{i,j}$ is normalized by subtracting μ_j

and dividing by σ_j . The rest of the network then operates on $\mathbf{H}'$ in exactly the same way that the original network operated on $\mathbf{H}$.

At training time,

$$\boldsymbol{\mu} = \frac{1}{m} \sum_i \mathbf{H}_{i,:} \quad (8.36)$$

and

$$\boldsymbol{\sigma} = \sqrt{\delta + \frac{1}{m} \sum_i (\mathbf{H} - \boldsymbol{\mu})_i^2}, \quad (8.37)$$

where δ is a small positive value such as 10^{-8} imposed to avoid encountering the undefined gradient of $\sqrt{z}$ at $z = 0$. Crucially, *we back-propagate through these operations* for computing the mean and the standard deviation, and for applying them to normalize $\mathbf{H}$. This means that the gradient will never propose an operation that acts simply to increase the standard deviation or mean of h_i ; the normalization operations remove the effect of such an action and zero out its component in the gradient. This was a major innovation of the batch normalization approach. Previous approaches had involved adding penalties to the cost function to encourage units to have normalized activation statistics or involved intervening to renormalize unit statistics after each gradient descent step. The former approach usually resulted in imperfect normalization and the latter usually resulted in significant wasted time as the learning algorithm repeatedly proposed changing the mean and variance and the normalization step repeatedly undid this change. Batch normalization reparametrizes the model to make some units always be standardized by definition, deftly sidestepping both problems.

At test time, $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ may be replaced by running averages that were collected during training time. This allows the model to be evaluated on a single example, without needing to use definitions of $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ that depend on an entire minibatch.

Revisiting the $\hat{y} = xw_1w_2 \dots w_l$ example, we see that we can mostly resolve the difficulties in learning this model by normalizing h_{l-1} . Suppose that x is drawn from a unit Gaussian. Then h_{l-1} will also come from a Gaussian, because the transformation from x to h_l is linear. However, h_{l-1} will no longer have zero mean and unit variance. After applying batch normalization, we obtain the normalized $\hat{h}_{l-1}$ that restores the zero mean and unit variance properties. For almost any update to the lower layers, $\hat{h}_{l-1}$ will remain a unit Gaussian. The output $\hat{y}$ may then be learned as a simple linear function $\hat{y} = w\hat{h}_{l-1}$. Learning in this model is now very simple because the parameters at the lower layers simply do not have an effect in most cases; their output is always renormalized to a unit Gaussian. In some corner cases, the lower layers can have an effect. Changing one of the lower layer weights to 0 can make the output become degenerate, and changing the sign

of one of the lower weights can flip the relationship between $\hat{h}_{l-1}$ and y . These situations are very rare. Without normalization, nearly every update would have an extreme effect on the statistics of h_{l-1} . Batch normalization has thus made this model significantly easier to learn. In this example, the ease of learning of course came at the cost of making the lower layers useless. In our linear example, the lower layers no longer have any harmful effect, but they also no longer have any beneficial effect. This is because we have normalized out the first and second order statistics, which is all that a linear network can influence. In a deep neural network with nonlinear activation functions, the lower layers can perform nonlinear transformations of the data, so they remain useful. Batch normalization acts to standardize only the mean and variance of each unit in order to stabilize learning, but allows the relationships between units and the nonlinear statistics of a single unit to change.

Because the final layer of the network is able to learn a linear transformation, we may actually wish to remove all linear relationships between units within a layer. Indeed, this is the approach taken by [Desjardins *et al.* \(2015\)](#), who provided the inspiration for batch normalization. Unfortunately, eliminating all linear interactions is much more expensive than standardizing the mean and standard deviation of each individual unit, and so far batch normalization remains the most practical approach.

Normalizing the mean and standard deviation of a unit can reduce the expressive power of the neural network containing that unit. In order to maintain the expressive power of the network, it is common to replace the batch of hidden unit activations $\mathbf{H}$ with $\gamma\mathbf{H}' + \beta$ rather than simply the normalized $\mathbf{H}'$. The variables γ and β are learned parameters that allow the new variable to have any mean and standard deviation. At first glance, this may seem useless—why did we set the mean to $\mathbf{0}$, and then introduce a parameter that allows it to be set back to any arbitrary value β ? The answer is that the new parametrization can represent the same family of functions of the input as the old parametrization, but the new parametrization has different learning dynamics. In the old parametrization, the mean of $\mathbf{H}$ was determined by a complicated interaction between the parameters in the layers below $\mathbf{H}$. In the new parametrization, the mean of $\gamma\mathbf{H}' + \beta$ is determined solely by β . The new parametrization is much easier to learn with gradient descent.

Most neural network layers take the form of $\phi(\mathbf{X}\mathbf{W} + \mathbf{b})$ where ϕ is some fixed nonlinear activation function such as the rectified linear transformation. It is natural to wonder whether we should apply batch normalization to the input $\mathbf{X}$, or to the transformed value $\mathbf{X}\mathbf{W} + \mathbf{b}$. [Ioffe and Szegedy \(2015\)](#) recommend